

SAKSHAMMATCH - COMPLETE IMPLEMENTATION REPORT

SIH26092: AI-Driven Scheme Matching for Marginalized Entrepreneurs

Ministry of Social Justice & Empowerment | Smart Automation Theme

TABLE OF CONTENTS

1. Executive Summary
2. Problem Understanding & Differentiation
3. Solution Architecture Overview
4. Frontend Implementation Plan
5. Backend Implementation Plan
6. Database Design & Connectivity
7. Web Scraping Strategy for Scheme Data
8. AI/ML Engine Implementation
9. GitHub Repository Setup
10. Deployment & Hosting
11. Testing & Quality Assurance
12. Timeline & Milestones
13. Skills & Tools Required
14. Budget & Resources
15. Risk Mitigation
16. Final Submission Checklist

SECTION 1: EXECUTIVE SUMMARY

1.1 Project Overview

- **Problem Statement Code:** SIH26092
- **Title:** AI-Driven Scheme Matching for Marginalized Entrepreneurs
- **Ministry:** Ministry of Social Justice & Empowerment (MoSJE)
- **Theme:** Smart Automation

- **Solution Name:** SakshamMatch

1.2 What We're Building

A web-based platform that:

- Takes entrepreneur profile (8-10 questions)
- Matches with eligible government schemes using AI
- Explains "why you qualify" for each scheme
- Provides document checklist and application links
- Targets SC/ST/OBC, women, rural entrepreneurs

1.3 Key Differentiators from Existing Solutions

Feature	myScheme	Startup India	SakshamMatch (Our Solution)
Target Users	Generic citizens	Formal startups	Marginalized entrepreneurs
Matching Logic	Rule-based filters	Manual browsing	AI + Rules + Explainability
Access Channels	Web only	Web only	Web + WhatsApp + CSC Dashboard
Language Support	Limited	English-heavy	Hindi + 10+ regional + Voice
Assisted Onboarding	No	No	Yes (CSC/NGO facilitators)
Analytics for Govt	No	No	Yes (MoSJE dashboard)

1.4 Implementation Approach

- **Phase 1 (Weeks 1-2):** Frontend + Backend MVP
- **Phase 2 (Weeks 3-4):** Web Scraping + Database Population
- **Phase 3 (Weeks 5-6):** AI/ML Engine + Testing
- **Phase 4 (Week 7):** Deployment + Final Polish

SECTION 2: PROBLEM UNDERSTANDING & DIFFERENTIATION

2.1 Core Problem

- 63M MSMEs employ 120M+ people but only 20% are women-owned
- 41% of SC/ST women unaware of government schemes
- 70% rural youth lack basic digital skills
- Schemes exist but fragmented across 10+ portals
- Complex eligibility rules in PDFs and bureaucratic language

2.2 Why Existing Solutions Fail

myScheme:

- Generic citizen focus, not entrepreneur-specific
- No AI/ML matching, only basic filters
- Text-heavy, not designed for low-literacy users
- No assisted onboarding

Startup India:

- Only for DPIIT-recognized formal startups
- Excludes informal, rural, first-generation entrepreneurs
- No personalized matching engine

Udyamimitra/Bank Portals:

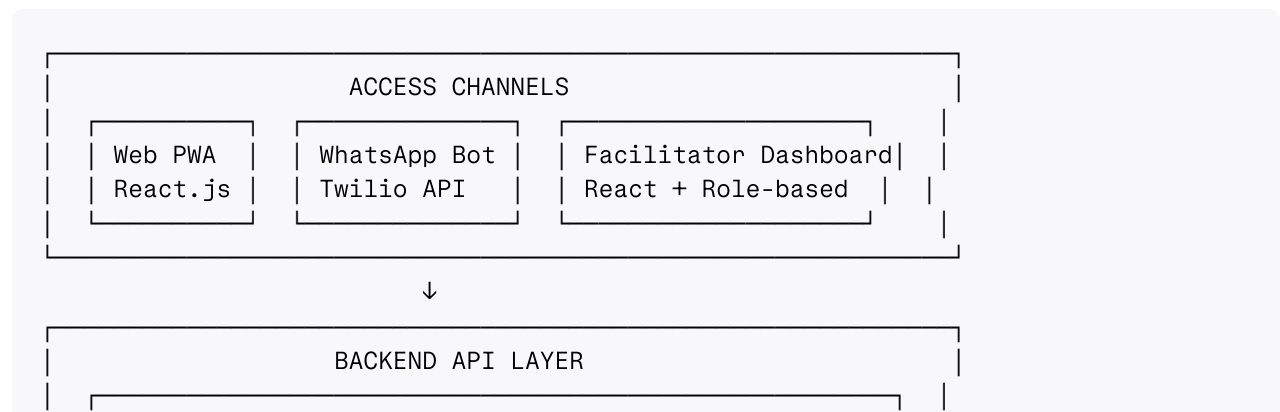
- Credit-focused (loans only)
- No grants, subsidies, or skill development
- Limited to MSME-registered entities

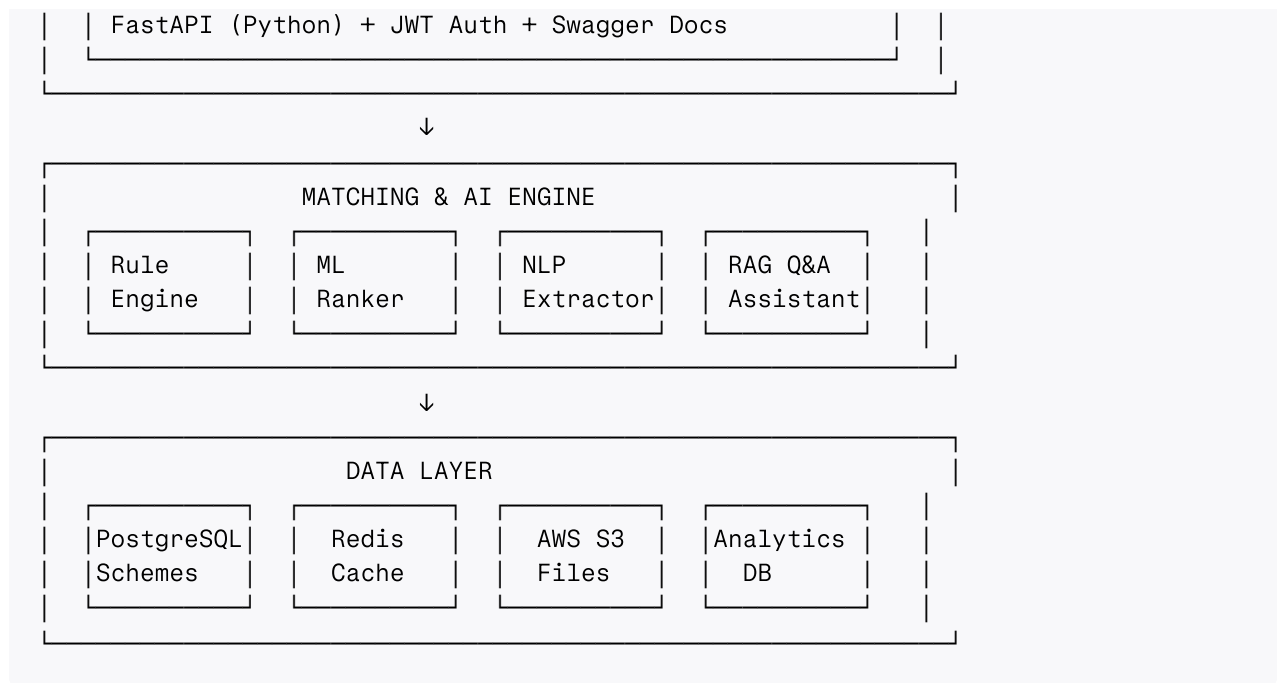
2.3 Our Unique Value Proposition

1. **Entrepreneur-Focused:** Built specifically for marginalized entrepreneurs (SC/ST/OBC, women, rural)
2. **AI-Driven Matching:** Not just filters - AI ranks and explains "why you qualify"
3. **Multi-Channel Access:** Web PWA + WhatsApp + CSC facilitator dashboard
4. **Low-Literacy Friendly:** Voice input, icons, vernacular languages
5. **Assisted Onboarding:** Human support via CSCs and NGOs
6. **Policy Insights:** Analytics dashboard for MoSJE to improve schemes

SECTION 3: SOLUTION ARCHITECTURE OVERVIEW

3.1 High-Level Architecture





3.2 Data Flow

1. User creates profile via Web PWA (8-10 questions)
2. Frontend sends profile to Backend API (FastAPI)
3. Backend calls Matching Engine:
 - Rule engine validates hard eligibility
 - ML ranker scores eligible schemes
 - NLP extracts scheme details from scraped data
4. Backend returns top 3-5 matched schemes with explanations
5. Frontend displays results with document checklist
6. User clicks "Apply" → redirected to official portal
7. All data stored in PostgreSQL with audit logs

3.3 Component Breakdown

Frontend (React + Next.js):

- User onboarding form
- Scheme matching results page
- Document checklist generator
- Application tracker
- Profile dashboard

Backend (FastAPI):

- User authentication (JWT)

- Profile management APIs
- Scheme matching APIs
- Document generation APIs
- Analytics APIs (for MoSJE)

Database (PostgreSQL):

- Users table
- Schemes table (scraped data)
- Applications table
- Facilitators table
- Analytics table

AI/ML Engine:

- Rule engine (eligibility validation)
- NLP pipeline (scheme extraction)
- ML ranker (XGBoost)
- Explainability module (SHAP)

SECTION 4: FRONTEND IMPLEMENTATION PLAN

4.1 Tech Stack

- **Framework:** React.js + Next.js (Progressive Web App)
- **UI Library:** Tailwind CSS + Headless UI
- **State Management:** Redux Toolkit
- **Form Handling:** React Hook Form + Zod validation
- **Charts:** Recharts (for analytics dashboard)
- **PDF Generation:** jsPDF + react-pdf
- **PWA Features:** Workbox (offline caching, push notifications)
- **Deployment:** Vercel (free tier)

4.2 Folder Structure

```
frontend/  
├── public/  
│   ├── manifest.json (PWA config)  
│   └── icons/ (app icons)  
├── src/  
│   ├── components/  
│   │   ├── Header.jsx  
│   │   └── Footer.jsx
```

```

|   |   |   | OnboardingForm.jsx
|   |   |   | SchemeCard.jsx
|   |   |   | DocumentChecklist.jsx
|   |   |   | ApplicationTracker.jsx
|   |   |   |
|   |   |   | pages/
|   |   |   | | index.js (landing page)
|   |   |   | | onboarding.js (profile form)
|   |   |   | | results.js (matched schemes)
|   |   |   | | dashboard.js (user profile)
|   |   |   | | admin.js (analytics)
|   |   |   |
|   |   |   | store/
|   |   |   | | index.js (Redux store)
|   |   |   |
|   |   |   | utils/
|   |   |   | | api.js (API calls)
|   |   |   | | validation.js (form validation)
|   |   |   |
|   |   |   | styles/
|   |   |   | | globals.css (Tailwind)
|   |   |   |
|   |   |   | package.json
|   |   |   | next.config.js
|   |   |   | tailwind.config.js

```

4.3 Implementation Steps

Step 1: Setup Next.js Project (Day 1)

- Install Node.js (LTS version)
- Run: `npx create-next-app@latest sakshammatch-frontend`
- Select: TypeScript, Tailwind CSS, App Router
- Install dependencies: Redux Toolkit, React Hook Form, Zod, Recharts, jsPDF

Step 2: Create Landing Page (Day 2)

- Design hero section with value proposition
- Add "Get Started" button → redirects to onboarding
- Add stats section (63M MSMEs, 30% GDP, etc.)
- Add footer with SIH26092 branding

Step 3: Build Onboarding Form (Days 3-4)

- Create 8-10 question form:
 1. Age
 2. Gender
 3. Category (SC/ST/OBC/General)
 4. State, District
 5. Rural/Urban
 6. Monthly Income
 7. Family Income

8. Business Type (Manufacturing/Service/Agri/Gig)

9. Business Stage (Idea/Registered/Revenue)

10. Education, Skills

- Add validation using Zod (required fields, number ranges)
- Store responses in Redux state
- On submit: POST to backend API /api/match

Step 4: Build Results Page (Days 5-6)

- Fetch matched schemes from backend
- Display top 3-5 schemes as cards
- Each card shows:
 - Scheme name
 - Benefit amount
 - Eligibility summary
 - "Why you qualify" explanation
 - "View Details" button
- Add "Download Checklist" button (generates PDF)
- Add "Apply Now" button (links to official portal)

Step 5: Build Document Checklist (Day 7)

- Create PDF template with:
 - User name, category
 - Matched scheme names
 - Required documents for each scheme
 - Application links
- Use jsPDF to generate downloadable PDF
- Add "Print" and "Save" options

Step 6: Build User Dashboard (Days 8-9)

- Show saved schemes
- Show application history (status: Not Applied → Applied → Approved)
- Allow profile editing
- Add logout button

Step 7: Build Admin Analytics (Days 10-11)

- Create charts:
 - Total users by category (SC/ST/OBC/Women)
 - Top schemes accessed

- District-wise uptake
- Drop-off points
- Use Recharts for visualization
- Add "Export as PDF" button

Step 8: PWA Configuration (Day 12)

- Add manifest.json (app name, icons, theme color)
- Configure Workbox for offline caching
- Test offline functionality
- Add to home screen capability

Step 9: Testing & Polish (Days 13-14)

- Test on mobile devices
- Fix responsive design issues
- Optimize loading speed
- Add error handling
- Final UI polish

4.4 Skills Required

- React.js + Next.js (intermediate)
- Tailwind CSS (basic)
- Redux Toolkit (basic)
- Form validation (basic)
- PDF generation (basic)
- PWA configuration (basic)

4.5 Tools & Resources

- VS Code (editor)
- Chrome DevTools (debugging)
- Figma (UI design reference)
- Tailwind UI (component templates)
- Vercel (deployment)

SECTION 5: BACKEND IMPLEMENTATION PLAN

5.1 Tech Stack

- **Framework:** FastAPI (Python)
- **Authentication:** JWT (PyJWT)
- **Database ORM:** SQLAlchemy
- **Caching:** Redis
- **Task Queue:** Celery (for background jobs)
- **File Storage:** AWS S3 (or local for MVP)
- **Email/SMS:** SendGrid / MSG91
- **API Docs:** Swagger (auto-generated by FastAPI)
- **Deployment:** Render / Railway (free tier)

5.2 Folder Structure

```
backend/
├── app/
│   ├── api/
│   │   ├── routes/
│   │   │   ├── auth.py (login, register)
│   │   │   ├── profile.py (profile CRUD)
│   │   │   ├── matching.py (scheme matching)
│   │   │   ├── documents.py (PDF generation)
│   │   │   └── analytics.py (MoSJE dashboard)
│   │   └── deps.py (dependencies, auth)
│   ├── core/
│   │   ├── config.py (environment variables)
│   │   ├── security.py (JWT, password hashing)
│   │   └── celery_app.py (Celery config)
│   ├── db/
│   │   ├── base.py (SQLAlchemy base)
│   │   ├── session.py (database session)
│   │   └── init_db.py (database initialization)
│   ├── models/
│   │   ├── user.py
│   │   ├── scheme.py
│   │   ├── application.py
│   │   └── facilitator.py
│   ├── schemas/
│   │   ├── user.py (Pydantic schemas)
│   │   ├── scheme.py
│   │   └── matching.py
│   ├── services/
│   │   ├── matching_engine.py (rule engine + ML ranker)
│   │   ├── nlp_extractor.py (scheme extraction)
│   │   └── pdf_generator.py (document checklist)
│   └── tasks/
│       └── background_jobs.py (Celery tasks)
```

```
├─ alembic/ (database migrations)
├─ requirements.txt
├─ .env (environment variables)
└─ main.py (FastAPI app entry)
```

5.3 Implementation Steps

Step 1: Setup FastAPI Project (Day 1)

- Install Python 3.10+
- Create virtual environment
- Install dependencies: FastAPI, SQLAlchemy, PyJWT, Celery, Redis, etc.
- Create folder structure as above
- Set up environment variables (.env file)

Step 2: Database Connection (Day 2)

- Configure SQLAlchemy to connect to PostgreSQL
- Create database session
- Write `init_db.py` to create tables on startup
- Test connection with simple query

Step 3: User Authentication (Days 3-4)

- Create User model (id, email, password_hash, created_at)
- Implement password hashing (bcrypt)
- Create JWT token generation and validation
- Build `/api/auth/register` and `/api/auth/login` endpoints
- Add authentication middleware (protect routes)

Step 4: Profile Management APIs (Days 5-6)

- Create Profile model (user_id, age, gender, category, income, etc.)
- Build CRUD APIs:
 - POST `/api/profile` (create profile)
 - GET `/api/profile` (get profile)
 - PUT `/api/profile` (update profile)
- Add validation (required fields, data types)

Step 5: Scheme Matching API (Days 7-9)

- Create Scheme model (id, name, ministry, benefits, eligibility_rules, etc.)
- Build `/api/match` endpoint:
 - Accept user profile as input

- Call Matching Engine service
- Return top 3-5 matched schemes with explanations
- Add caching (Redis) for frequent queries

Step 6: Matching Engine Service (Days 10-12)

- Implement Rule Engine:
 - Load eligibility rules from database
 - Validate user profile against rules
 - Return list of eligible schemes
- Implement ML Ranker:
 - Load pre-trained XGBoost model
 - Score eligible schemes by relevance
 - Return ranked list
- Implement Explainability:
 - For each matched scheme, list which rules matched
 - Generate "You qualify because..." text

Step 7: Document Generation API (Day 13)

- Build `/api/documents/checklist` endpoint
- Accept `user_id` and `scheme_ids`
- Generate PDF with:
 - User details
 - Matched scheme names
 - Required documents for each
 - Application links
- Return PDF as downloadable file

Step 8: Analytics API (Days 14-15)

- Build `/api/analytics/overview` endpoint
- Return:
 - Total users by category
 - Top schemes accessed
 - District-wise uptake
 - Application conversion rates
- Add authentication (only MoSJE admins can access)

Step 9: Background Jobs (Day 16)

- Set up Celery with Redis

- Create tasks:
 - Send email notifications
 - Send SMS reminders
 - Generate PDF reports asynchronously
- Configure Celery Beat for scheduled tasks

Step 10: API Documentation (Day 17)

- FastAPI auto-generates Swagger docs at /docs
- Test all endpoints using Swagger UI
- Add request/response examples
- Add authentication examples

Step 11: Testing & Optimization (Days 18-19)

- Write unit tests for each API
- Test with Postman
- Optimize database queries (add indexes)
- Add error handling
- Final polish

5.4 Skills Required

- FastAPI (intermediate)
- SQLAlchemy ORM (intermediate)
- JWT authentication (basic)
- PostgreSQL (basic)
- Redis (basic)
- Celery (basic)
- REST API design (intermediate)

5.5 Tools & Resources

- PyCharm or VS Code (editor)
- Postman (API testing)
- Swagger UI (documentation)
- Render/Railway (deployment)

SECTION 6: DATABASE DESIGN & CONNECTIVITY

6.1 Database Tech Stack

- **Primary Database:** PostgreSQL 15+
- **ORM:** SQLAlchemy (Python)
- **Migrations:** Alembic
- **Caching:** Redis
- **Hosting:** Supabase (free tier) or Neon (free tier) or AWS RDS

6.2 Database Schema

Table 1: users

- id (UUID, primary key)
- email (string, unique)
- password_hash (string)
- created_at (timestamp)
- updated_at (timestamp)
- is_active (boolean)

Table 2: profiles

- id (UUID, primary key)
- user_id (UUID, foreign key → users.id)
- age (integer)
- gender (enum: Male/Female/Other)
- category (enum: SC/ST/OBC/General)
- state (string)
- district (string)
- rural_urban (enum: Rural/Urban)
- monthly_income (decimal)
- family_income (decimal)
- business_type (enum: Manufacturing/Service/Agri/Gig)
- business_stage (enum: Idea/Registered/Revenue)
- education (string)
- skills (text)
- created_at (timestamp)
- updated_at (timestamp)

Table 3: schemes

- id (UUID, primary key)
- name (string)
- ministry (string)
- description (text)
- benefits (text)
- benefit_amount (decimal)
- eligibility_rules (JSONB)

- documents_required (JSONB)
- application_link (string)
- source_url (string)
- is_active (boolean)
- scraped_at (timestamp)
- created_at (timestamp)
- updated_at (timestamp)

Table 4: applications

- id (UUID, primary key)
- user_id (UUID, foreign key → users.id)
- scheme_id (UUID, foreign key → schemes.id)
- status (enum: Not Applied/Applied/Approved/Rejected)
- applied_at (timestamp)
- approved_at (timestamp)
- documents_submitted (JSONB)
- notes (text)
- created_at (timestamp)
- updated_at (timestamp)

Table 5: facilitators

- id (UUID, primary key)
- name (string)
- email (string)
- organization (string)
- csc_id (string)
- state (string)
- district (string)
- is_active (boolean)
- created_at (timestamp)

Table 6: facilitator_beneficiaries

- id (UUID, primary key)
- facilitator_id (UUID, foreign key → facilitators.id)
- user_id (UUID, foreign key → users.id)
- assigned_at (timestamp)
- status (enum: Active/Completed)

Table 7: analytics

- id (UUID, primary key)
- date (date)
- total_users (integer)
- new_users (integer)
- total_applications (integer)
- approved_applications (integer)
- top_scheme_id (UUID, foreign key → schemes.id)

- district (string)
- category (string)

6.3 Implementation Steps

Step 1: Database Setup (Day 1)

- Create Supabase/Neon account (free tier)
- Create new PostgreSQL database
- Get connection string
- Add to .env file

Step 2: SQLAlchemy Configuration (Day 2)

- Create `db/session.py` with database session
- Create `db/base.py` with SQLAlchemy base class
- Test connection with simple query

Step 3: Create Models (Days 3-4)

- Create all 7 models in `models/` folder
- Define relationships (foreign keys)
- Add indexes on frequently queried fields (`user_id`, `scheme_id`, `category`, `district`)

Step 4: Database Migrations (Day 5)

- Initialize Alembic: `alembic init alembic`
- Configure `alembic.ini` with database URL
- Create first migration: `alembic revision --autogenerate -m "initial"`
- Run migration: `alembic upgrade head`
- Verify tables created in database

Step 5: Seed Initial Data (Day 6)

- Create `seed_data.py` script
- Insert sample users (for testing)
- Insert sample schemes (from web scraping)
- Run script to populate database

Step 6: Database Connectivity (Day 7)

- Test CRUD operations for each model
- Verify foreign key relationships
- Test indexes for performance
- Add connection pooling configuration

Step 7: Redis Caching (Day 8)

- Set up Redis (local or Redis Cloud free tier)
- Cache frequently accessed schemes
- Cache user sessions
- Add cache invalidation logic

6.4 Skills Required

- PostgreSQL (basic)
- SQLAlchemy ORM (intermediate)
- Database design (basic)
- Alembic migrations (basic)
- Redis (basic)

6.5 Tools & Resources

- pgAdmin or DBeaver (database GUI)
- Supabase/Neon (hosting)
- Redis Cloud (caching)

SECTION 7: WEB SCRAPING STRATEGY FOR SCHEME DATA

7.1 Data Sources

Primary Sources:

1. data.gov.in - Open government data platform
2. myscheme.gov.in - Central scheme repository
3. **Ministry Websites:**
 - Ministry of MSME (msme.gov.in)
 - Ministry of Social Justice (socialjustice.gov.in)
 - Ministry of Rural Development (ruraldev.gov.in)
 - NITI Aayog (niti.gov.in)
4. **State MSME Portals:**
 - Karnataka (kassia.kar.nic.in)
 - Tamil Nadu (msme.tn.gov.in)
 - Maharashtra (msme.maharashtra.gov.in)
5. **Bank Portals:**
 - MUDRA (mudra.org.in)

- Stand-Up India (standupmitra.in)

7.2 Scraping Tech Stack

- **Framework:** Scrapy or BeautifulSoup + Requests
- **Browser Automation:** Selenium or Playwright (for JavaScript-heavy sites)
- **Data Storage:** PostgreSQL (schemes table)
- **Scheduling:** Celery Beat (run scraper daily)
- **Proxy:** Free proxies or ScraperAPI (if rate-limited)
- **Data Cleaning:** Pandas (Python)

7.3 Implementation Steps

Step 1: Identify Target URLs (Day 1)

- List all government portals with scheme data
- Identify scheme listing pages
- Identify individual scheme detail pages
- Create spreadsheet with URLs

Step 2: Analyze Page Structure (Day 2)

- Inspect HTML structure of target pages
- Identify scheme name, description, benefits, eligibility selectors
- Check if data is in tables, cards, or unstructured text
- Note if JavaScript is required to load data

Step 3: Build Basic Scraper (Days 3-4)

- Install Scrapy or BeautifulSoup
- Create scraper script:
 - Fetch scheme listing page
 - Extract scheme names and links
 - For each scheme, fetch detail page
 - Extract: name, ministry, description, benefits, eligibility, documents, apply link
- Store extracted data as JSON

Step 4: Handle JavaScript-Heavy Sites (Day 5)

- For sites that load data via JavaScript:
 - Use Selenium or Playwright
 - Wait for page to fully load
 - Extract data from rendered HTML

- Add delays to avoid rate-limiting

Step 5: Data Cleaning & Validation (Days 6-7)

- Use Pandas to clean extracted data:
 - Remove duplicates
 - Standardize field names
 - Validate data types (benefit_amount should be numeric)
 - Fill missing values where possible
- Validate against manual samples

Step 6: Load into Database (Day 8)

- Create data loading script:
 - Read cleaned JSON/CSV
 - Insert into PostgreSQL schemes table
 - Handle duplicates (check by name + ministry)
 - Add source_url and scraped_at timestamp
- Verify data in database

Step 7: Schedule Regular Scraping (Day 9)

- Set up Celery Beat task:
 - Run scraper daily at 2 AM
 - Update existing schemes (if data changed)
 - Insert new schemes
 - Log scraping results
- Add error handling and alerts

Step 8: Manual Verification (Day 10)

- Manually verify 10-20 scraped schemes
- Check accuracy of extracted data
- Fix scraper logic if needed
- Document data quality issues

7.4 Sample Data to Extract

For each scheme, extract:

```
{
  "name": "PMEGP",
  "ministry": "Ministry of MSME",
  "description": "Prime Minister's Employment Generation Programme",
  "benefits": "Subsidy up to 35% of project cost",
```

```

"benefit_amount": 500000,
"eligibility_rules": {
  "age_min": 18,
  "age_max": null,
  "category": ["SC", "ST", "OBC", "General"],
  "income_ceiling": null,
  "business_type": ["Manufacturing", "Service"],
  "business_stage": ["New"],
  "location": null
},
"documents_required": [
  "Aadhaar Card",
  "Caste Certificate (if applicable)",
  "Income Certificate",
  "Business Plan",
  "Udyam Registration"
],
"application_link": "https://pmegp.msme.gov.in/",
"source_url": "https://pmegp.msme.gov.in/Home/About",
"scraped_at": "2026-08-29T10:00:00Z"
}

```

7.5 Skills Required

- Web scraping (intermediate)
- HTML/CSS (basic)
- Python (intermediate)
- Data cleaning (basic)
- APIs (basic)

7.6 Tools & Resources

- Scrapy or BeautifulSoup
- Selenium/Playwright
- Pandas (data cleaning)
- Postman (API testing)
- Proxy services (if needed)

SECTION 8: AI/ML ENGINE IMPLEMENTATION

8.1 AI/ML Tech Stack

- **NLP:** spaCy + Hugging Face Transformers
- **ML Framework:** Scikit-learn + XGBoost
- **Explainability:** SHAP + LIME
- **RAG:** LangChain + ChromaDB

- **Model Serving:** FastAPI + Uvicorn
- **Training:** Google Colab (free GPU)

8.2 Components

Component 1: Rule Engine

- Validates hard eligibility criteria
- Returns list of eligible schemes
- Deterministic (no ML)

Component 2: NLP Extractor

- Extracts eligibility rules from scraped scheme text
- Uses spaCy for entity recognition
- Uses Hugging Face for text classification

Component 3: ML Ranker

- Scores eligible schemes by relevance
- Uses XGBoost for ranking
- Features: user profile, scheme benefits, past success rates

Component 4: Explainability Module

- Generates "You qualify because..." text
- Uses SHAP to identify important features
- Returns human-readable explanations

Component 5: RAG Q&A Assistant

- Answers user questions about schemes
- Uses LangChain for retrieval
- Uses ChromaDB for vector storage
- Returns answers with source citations

8.3 Implementation Steps

Step 1: Rule Engine (Days 1-2)

- Define eligibility rule schema (JSON)
- Load rules from database
- Implement validation logic:
 - Check age range
 - Check category (SC/ST/OBC)
 - Check income ceiling

- Check business type
- Check location
- Return list of eligible schemes

Step 2: NLP Extractor (Days 3-5)

- Install spaCy + Hugging Face
- Create entity recognition pipeline:
 - Extract age limits
 - Extract income ceilings
 - Extract category requirements
 - Extract location requirements
- Train on manually labeled samples (50-100 schemes)
- Evaluate accuracy (target: 85%+)

Step 3: ML Ranker (Days 6-10)

- Create training dataset:
 - User profiles (synthetic)
 - Scheme features (from database)
 - Labels: relevance score (1-5)
- Train XGBoost model:
 - Features: user age, income, category, business type, scheme benefits, etc.
 - Target: relevance score
- Evaluate model (target: 80%+ accuracy)
- Save model as .pkl file

Step 4: Explainability Module (Days 11-12)

- Install SHAP
- For each matched scheme:
 - Calculate SHAP values for features
 - Identify top 3 reasons for match
 - Generate text: "You qualify because: [reason 1], [reason 2], [reason 3]"
- Test with sample profiles

Step 5: RAG Q&A Assistant (Days 13-16)

- Install LangChain + ChromaDB
- Create embeddings for scheme documents:
 - Use Hugging Face embedding model
 - Store in ChromaDB

- Build retrieval pipeline:
 - User asks question
 - Retrieve top 3 relevant scheme sections
 - Generate answer with citations
- Test with sample questions

Step 6: Model Deployment (Days 17-18)

- Create FastAPI endpoints:
 - POST /api/ai/match (matching engine)
 - POST /api/ai/explain (explainability)
 - POST /api/ai/ask (Q&A assistant)
- Load pre-trained models on startup
- Test with sample requests

Step 7: Testing & Optimization (Days 19-20)

- Test with real user profiles
- Measure accuracy (target: 85%+)
- Optimize model (reduce latency)
- Add caching for frequent queries

8.4 Skills Required

- NLP (intermediate)
- Machine Learning (intermediate)
- XGBoost (basic)
- SHAP (basic)
- LangChain (basic)
- Python (intermediate)

8.5 Tools & Resources

- Google Colab (free GPU for training)
- Hugging Face (pre-trained models)
- spaCy (NLP)
- LangChain (RAG)
- ChromaDB (vector database)

SECTION 9: GITHUB REPOSITORY SETUP

9.1 Repository Structure (Monorepo)

```
sakshammatch-sih26092/
├── README.md (project overview, setup instructions)
├── .gitignore (Python, Node, IDE files)
├── LICENSE (MIT)
├── docs/
│   ├── IMPLEMENTATION.md (this report)
│   ├── API_DOCS.md (API documentation)
│   ├── DATABASE_SCHEMA.md (database design)
│   └── DEPLOYMENT.md (deployment guide)
├── frontend/
│   ├── src/
│   ├── public/
│   ├── package.json
│   └── README.md (frontend setup)
├── backend/
│   ├── app/
│   ├── alembic/
│   ├── requirements.txt
│   └── README.md (backend setup)
├── scraper/
│   ├── spiders/
│   ├── requirements.txt
│   └── README.md (scraper setup)
├── ml_models/
│   ├── training/
│   ├── models/
│   ├── requirements.txt
│   └── README.md (ML setup)
├── database/
│   ├── schema.sql
│   ├── seed_data.sql
│   └── README.md (database setup)
└── deployment/
    ├── docker-compose.yml
    ├── Dockerfile.frontend
    ├── Dockerfile.backend
    └── README.md (deployment setup)
```

9.2 Implementation Steps

Step 1: Create GitHub Repository (Day 1)

- Go to github.com
- Create new repo: sakshammatch-sih26092
- Make it public
- Add MIT License
- Add .gitignore (Python + Node)

Step 2: Initialize Local Repo (Day 1)

- Create project folder
- Run: `git init`
- Add remote: `git remote add origin <your-repo-url>`
- Create folder structure as above

Step 3: Create README.md (Day 2)

- Add project title
- Add problem statement
- Add solution overview
- Add tech stack
- Add setup instructions
- Add team members
- Add SIH26092 branding

Step 4: Create .gitignore (Day 2)

- Add Python patterns (`*.pyc`, `pycache`, `.env`)
- Add Node patterns (`node_modules`, `.next`)
- Add IDE patterns (`.vscode`, `.idea`)
- Add database patterns (`*.db`, `*.sqlite`)

Step 5: Create Documentation (Days 3-4)

- Copy this implementation report to `docs/IMPLEMENTATION.md`
- Create `docs/API_DOCS.md` (API endpoints)
- Create `docs/DATABASE_SCHEMA.md` (database design)
- Create `docs/DEPLOYMENT.md` (deployment guide)

Step 6: Commit Initial Structure (Day 5)

- Add all files: `git add .`
- Commit: `git commit -m "Initial commit: project structure"`
- Push: `git push -u origin main`

Step 7: Branch Strategy (Ongoing)

- `main`: Production-ready code
- `develop`: Integration branch
- `feature/*`: Individual features (e.g., `feature/onboarding-form`)
- Create PRs from feature branches to `develop`
- Merge `develop` to `main` when ready

Step 8: Contribution Guidelines (Day 6)

- Create `CONTRIBUTING.md`
- Define commit message format
- Define PR review process
- Define code style guidelines

9.3 Skills Required

- Git (basic)
- GitHub (basic)
- Markdown (basic)

9.4 Tools & Resources

- GitHub Desktop or Git CLI
- VS Code (with Git extension)

SECTION 10: DEPLOYMENT & HOSTING

10.1 Deployment Tech Stack

- **Frontend:** Vercel (free tier)
- **Backend:** Render or Railway (free tier)
- **Database:** Supabase or Neon (free tier)
- **Redis:** Redis Cloud (free tier)
- **File Storage:** AWS S3 (free tier for 12 months)
- **Domain:** Free subdomain (or buy custom domain)

10.2 Implementation Steps

Step 1: Frontend Deployment (Day 1)

- Create Vercel account
- Connect GitHub repo
- Select `frontend/` folder as root
- Configure build settings:
 - Build Command: `npm run build`
 - Output Directory: `.next`
- Deploy
- Get deployment URL (e.g., `sakshammatch.vercel.app`)

Step 2: Backend Deployment (Day 2)

- Create Render/Railway account
- Connect GitHub repo
- Select backend/ folder as root
- Configure environment variables:
 - DATABASE_URL
 - REDIS_URL
 - JWT_SECRET
 - etc.
- Deploy
- Get deployment URL (e.g., sakshamatch.onrender.com)

Step 3: Database Setup (Day 3)

- Create Supabase/Neon account
- Create new PostgreSQL database
- Get connection string
- Add to backend environment variables
- Run migrations: `alembic upgrade head`

Step 4: Redis Setup (Day 4)

- Create Redis Cloud account (free tier)
- Get Redis connection URL
- Add to backend environment variables
- Test connection

Step 5: File Storage Setup (Day 5)

- Create AWS account (free tier)
- Create S3 bucket
- Get access keys
- Add to backend environment variables
- Test file upload

Step 6: Environment Variables (Day 6)

- Create `.env` file for backend:

```
DATABASE_URL=postgresql://...  
REDIS_URL=redis://...  
JWT_SECRET=your-secret-key  
AWS_ACCESS_KEY=...
```

```
AWS_SECRET_KEY=...  
AWS_S3_BUCKET=...
```

- Add to deployment platform (Vercel/Render)
- Never commit .env to GitHub

Step 7: Custom Domain (Optional, Day 7)

- Buy domain (e.g., sakshammatch.in)
- Configure DNS:
 - Frontend: CNAME to Vercel
 - Backend: CNAME to Render
- Add SSL (auto-provided by Vercel/Render)

Step 8: Testing (Day 8)

- Test frontend on deployed URL
- Test backend APIs using Swagger
- Test database connectivity
- Test file uploads
- Fix any deployment issues

10.3 Skills Required

- Vercel (basic)
- Render/Railway (basic)
- Supabase/Neon (basic)
- AWS S3 (basic)
- Environment variables (basic)

10.4 Tools & Resources

- Vercel dashboard
- Render/Railway dashboard
- Supabase/Neon dashboard
- AWS Console

SECTION 11: TESTING & QUALITY ASSURANCE

11.1 Testing Strategy

Unit Testing:

- Test individual functions
- Test API endpoints
- Test database queries
- Tools: Pytest (backend), Jest (frontend)

Integration Testing:

- Test frontend + backend integration
- Test API + database integration
- Test AI/ML engine integration

User Acceptance Testing:

- Test with real user profiles
- Test end-to-end flow (onboarding → matching → application)
- Get feedback from 5-10 users

Performance Testing:

- Test API response times (target: <500ms)
- Test database query performance
- Test under load (100 concurrent users)

11.2 Implementation Steps

Step 1: Write Unit Tests (Days 1-3)

- Backend: Write tests for each API endpoint
- Frontend: Write tests for each component
- Database: Write tests for each model
- Aim for 80%+ code coverage

Step 2: Write Integration Tests (Days 4-5)

- Test frontend → backend → database flow
- Test matching engine end-to-end
- Test document generation flow

Step 3: User Acceptance Testing (Days 6-7)

- Create 10 test user profiles (diverse categories)
- Test onboarding flow
- Test matching results

- Test document checklist
- Collect feedback

Step 4: Performance Testing (Day 8)

- Use Apache Bench or k6
- Test API endpoints under load
- Measure response times
- Identify bottlenecks

Step 5: Bug Fixing (Days 9-10)

- Fix all critical bugs
- Fix major bugs
- Document minor bugs for future fixes

Step 6: Final QA (Day 11)

- Test on multiple devices (desktop, mobile, tablet)
- Test on multiple browsers (Chrome, Firefox, Safari)
- Test offline functionality (PWA)
- Test accessibility (screen readers)

11.3 Skills Required

- Pytest (basic)
- Jest (basic)
- Postman (basic)
- Apache Bench (basic)

11.4 Tools & Resources

- Pytest (Python testing)
- Jest (JavaScript testing)
- Postman (API testing)
- Chrome DevTools (debugging)

SECTION 12: TIMELINE & MILESTONES

12.1 Week-by-Week Breakdown

Week 1 (Days 1-7): Setup & Frontend

- Day 1: GitHub repo, folder structure
- Day 2-3: Frontend setup, landing page
- Day 4-5: Onboarding form
- Day 6-7: Results page

Week 2 (Days 8-14): Backend & Database

- Day 8-9: Backend setup, authentication
- Day 10-11: Profile APIs
- Day 12-13: Matching API
- Day 14: Database setup

Week 3 (Days 15-21): Web Scraping & AI/ML

- Day 15-16: Scraper setup
- Day 17-18: Data cleaning
- Day 19-20: Rule engine
- Day 21: NLP extractor

Week 4 (Days 22-28): ML Ranker & Testing

- Day 22-24: ML ranker training
- Day 25-26: Explainability module
- Day 27-28: Integration testing

Week 5 (Days 29-35): Deployment & Polish

- Day 29-30: Frontend deployment
- Day 31-32: Backend deployment
- Day 33-34: Testing & bug fixing
- Day 35: Final polish

Week 6 (Days 36-42): Documentation & Submission

- Day 36-37: API documentation
- Day 38-39: User manual
- Day 40-41: Demo video
- Day 42: Final submission

12.2 Critical Path

1. Frontend onboarding form (Week 1)
2. Backend matching API (Week 2)
3. Web scraping (Week 3)
4. ML ranker (Week 4)
5. Deployment (Week 5)

12.3 Buffer Time

- Add 2-3 days buffer for unexpected issues
- Prioritize MVP features first
- Defer nice-to-have features if behind schedule

SECTION 13: SKILLS & TOOLS REQUIRED

13.1 Skills Required by Team Member

Frontend Developer (1 person):

- React.js + Next.js (intermediate)
- Tailwind CSS (basic)
- Redux Toolkit (basic)
- PWA configuration (basic)
- Time: 20-25 hours/week

Backend Developer (1 person):

- FastAPI (intermediate)
- SQLAlchemy (intermediate)
- PostgreSQL (basic)
- Redis (basic)
- Time: 20-25 hours/week

Full-Stack/ML Developer (1 person):

- Web scraping (intermediate)
- NLP (basic)
- XGBoost (basic)
- Deployment (basic)
- Time: 20-25 hours/week

13.2 Tools & Software

Development:

- VS Code (editor)
- Git + GitHub (version control)
- Postman (API testing)
- Chrome DevTools (debugging)

Design:

- Figma (UI design)
- Tailwind UI (component templates)

Deployment:

- Vercel (frontend)
- Render/Railway (backend)
- Supabase/Neon (database)
- Redis Cloud (caching)

Communication:

- Discord/Slack (team chat)
- Notion/Trello (project management)
- Google Meet (video calls)

13.3 Learning Resources

Frontend:

- Next.js documentation (nextjs.org/docs)
- Tailwind CSS documentation (tailwindcss.com/docs)
- Redux Toolkit tutorial (redux-toolkit.js.org)

Backend:

- FastAPI documentation (fastapi.tiangolo.com)
- SQLAlchemy documentation (sqlalchemy.org)
- PostgreSQL tutorial (postgresqltutorial.com)

Web Scraping:

- Scrapy documentation (scrapy.org)
- BeautifulSoup tutorial (beautifulsoup.readthedocs.io)

AI/ML:

- Hugging Face course (huggingface.co/course)

- XGBoost documentation (xgboost.readthedocs.io)
- LangChain documentation (python.langchain.com)

SECTION 14: BUDGET & RESOURCES

14.1 Free Tier Tools

Frontend:

- Vercel: Free (unlimited projects)
- Next.js: Free (open-source)
- Tailwind CSS: Free (open-source)

Backend:

- Render: Free (500 hours/month)
- Railway: Free (\$5 credit/month)
- FastAPI: Free (open-source)

Database:

- Supabase: Free (500MB database)
- Neon: Free (500MB storage)
- PostgreSQL: Free (open-source)

Redis:

- Redis Cloud: Free (30MB)

File Storage:

- AWS S3: Free (5GB for 12 months)

AI/ML:

- Google Colab: Free (GPU for training)
- Hugging Face: Free (pre-trained models)

Total Cost: ₹0 (all free tiers)

14.2 Optional Paid Tools

If You Need More Resources:

- Vercel Pro: ₹1,600/month (more bandwidth)
- Render Standard: ₹500/month (no sleep)
- Supabase Pro: ₹800/month (more database)
- AWS S3: ₹100/month (after free tier)

- Custom Domain: ₹800/year

Total Optional Cost: ₹3,000-4,000/month

14.3 Team Resources

Time Commitment:

- 20-25 hours/week per person
- 6 weeks total
- Total: 360-450 hours (3 people)

Skills Investment:

- Learn new technologies
- Build portfolio project
- SIH26092 submission

SECTION 15: RISK MITIGATION

15.1 Technical Risks

Risk 1: Web Scraping Blocked

- Probability: Medium
- Impact: High
- Mitigation:
 - Use proxies
 - Add delays between requests
 - Fallback to manual curation
 - Use official APIs where available

Risk 2: AI/ML Model Inaccurate

- Probability: Medium
- Impact: High
- Mitigation:
 - Start with rule-based matching (deterministic)
 - Add ML as enhancement
 - Human review for edge cases
 - Continuous retraining with feedback

Risk 3: Deployment Issues

- Probability: Low

- Impact: Medium
- Mitigation:
 - Test locally first
 - Use free tiers for MVP
 - Have backup deployment options
 - Document deployment steps

15.2 Timeline Risks

Risk 1: Behind Schedule

- Probability: High
- Impact: Medium
- Mitigation:
 - Prioritize MVP features
 - Defer nice-to-have features
 - Add buffer time (2-3 days)
 - Daily standups to track progress

Risk 2: Team Member Drops Out

- Probability: Low
- Impact: High
- Mitigation:
 - Document everything
 - Cross-train team members
 - Have backup contributors
 - Keep code modular

15.3 Data Risks

Risk 1: Inaccurate Scheme Data

- Probability: Medium
- Impact: High
- Mitigation:
 - Manual verification of top 50 schemes
 - Source URLs for every scheme
 - Versioning (track data changes)
 - User feedback mechanism

Risk 2: Outdated Data

- Probability: Medium
- Impact: Medium
- Mitigation:
 - Daily scraping
 - Update timestamps
 - Notify users of data freshness
 - Link to official sources

SECTION 16: FINAL SUBMISSION CHECKLIST

16.1 Pre-Submission Checklist (1 Week Before Deadline)

Code:

- ☐ All features implemented
- ☐ No critical bugs
- ☐ Code reviewed by team
- ☐ README.md complete
- ☐ .gitignore configured
- ☐ No sensitive data in repo

Documentation:

- ☐ API documentation complete
- ☐ Database schema documented
- ☐ Deployment guide written
- ☐ User manual created
- ☐ SIH26092 PPT ready

Deployment:

- ☐ Frontend deployed and working
- ☐ Backend deployed and working
- ☐ Database connected
- ☐ All environment variables set
- ☐ SSL certificates configured

Testing:

- ☐ Unit tests passing (80%+ coverage)

- ☐ Integration tests passing
- ☐ User acceptance testing done
- ☐ Performance tests passing
- ☐ Cross-browser testing done

Submission:

- ☐ GitHub repo link ready
- ☐ Live demo link ready
- ☐ PPT uploaded
- ☐ Demo video recorded (optional)
- ☐ Team details submitted

16.2 Demo Preparation

Demo Script:

1. Show landing page (10 seconds)
2. Create test profile (30 seconds)
3. Show matched schemes (30 seconds)
4. Show document checklist (20 seconds)
5. Show application tracker (20 seconds)
6. Show analytics dashboard (30 seconds)
7. Total: 2-3 minutes

Demo Video:

- Record screen with OBS Studio (free)
- Add voiceover explaining features
- Edit with DaVinci Resolve (free)
- Upload to YouTube (unlisted) or Google Drive
- Add link to submission

16.3 Submission Day

Final Checks:

- ☐ All links working
- ☐ Demo video accessible
- ☐ PPT formatted correctly
- ☐ Team details correct
- ☐ Submitted before deadline

After Submission:

- Celebrate! 🎉
- Prepare for Q&A round
- Practice pitch presentation
- Be ready for prototype demo

CONCLUSION

This implementation report provides a complete, step-by-step guide to building SakshamMatch for SIH26092. Follow the sections in order, and you'll have a working prototype ready for submission.

Key Takeaways:

1. Start with MVP (web PWA + basic matching)
2. Add AI/ML as enhancement (not core)
3. Use free tiers for everything (₹0 cost)
4. Document everything (GitHub, README, docs)
5. Test thoroughly before submission
6. Focus on differentiation from existing solutions

Good luck with SIH26092! 🍀

APPENDIX: QUICK REFERENCE

A.1 Tech Stack Summary

Component	Technology
Frontend	React + Next.js + Tailwind CSS
Backend	FastAPI + SQLAlchemy
Database	PostgreSQL + Redis
AI/ML	spaCy + XGBoost + LangChain
Deployment	Vercel + Render + Supabase
Version Control	Git + GitHub

A.2 Folder Structure Summary

```
sakshammatch-sih26092/
├─ frontend/ (React + Next.js)
├─ backend/ (FastAPI + SQLAlchemy)
├─ scraper/ (Scrapy + BeautifulSoup)
├─ ml_models/ (XGBoost + LangChain)
├─ database/ (schema + seed data)
└─ deployment/ (Docker + configs)
```

A.3 Timeline Summary

Week	Focus
1	Frontend setup + onboarding
2	Backend + database
3	Web scraping + rule engine
4	ML ranker + testing
5	Deployment + polish
6	Documentation + submission

A.4 Team Roles Summary

Role	Person	Skills	Time/Week
Frontend Dev	[Name]	React, Next.js, Tailwind	20-25 hrs
Backend Dev	[Name]	FastAPI, SQLAlchemy, PostgreSQL	20-25 hrs
Full-Stack/ML	[Name]	Scraping, NLP, XGBoost	20-25 hrs

END OF IMPLEMENTATION REPORT